

Web Application Penetration Testing Report

COMMONWEALTH BANK INTRODUCTION TO CYBERSECURITY

Job Simulation

EXECUTIVE SUMMARY

As part of this project, 11 beginner-level web security labs were completed on the PortSwigger Web Security Academy platform. The objective was to identify and exploit common web application vulnerabilities in a safe, controlled environment and document the findings in a professional penetration testing report.

The labs covered a range of vulnerabilities, including SQL injection, cross-site scripting (XSS), authentication flaws, access control issues, path traversal, and command injections. Each vulnerability was successfully identified and exploited, demonstrating the potential impact that these issues can have on web applications. Overall, the labs highlighted the importance of secure coding practices, input validation, proper access controls, and defense in-depth strategies to reduce the risk of exploitation.

KEY TAKEAWAYS

- Common web vulnerabilities can be exploited with relatively simple techniques.
- Proper input validation, parameterized queries, and secure configuration significantly reduce risk.
- Regular security testing and code reviews are essential for detecting and remediating vulnerabilities.
- Even basic vulnerabilities can lead to serious impacts such as unauthorized access, data exposure, or complete system compromise.

SCOPE

This assessment was conducted using the PortSwigger Web Security Academy beginner level labs. These labs are intentionally vulnerable web applications designed for educational purposes.

Platform	PortSwigger Web Security Academy
Lab Level	Beginner
Total Labs Completed	11
Target Environment	Web application labs (Simulated)
Testing Approach	Manual testing using browser, Burp Suite, and common web exploitation techniques
Out of Scope	Real systems, production environments, or any unauthorized testing

OBJECTIVES

- Identify and exploit the intended vulnerability in each lab.
- Understand the potential impact of each vulnerability.
- Provide recommendations to improve the security of the web applications.
- Document findings and evidence (screenshots) for each lab.

NOTE

All testing was performed in a legal and authorized lab environment in accordance with PortSwigger's terms of service and ethical guidelines.

SUMMARY OF FINDINGS

The table below provides an overview of the 11 labs completed, the vulnerability type for each, and the potential impact.

Lab#	Lab Title/Vulnerability	Impact
1	SQL injection: Product retrieval	Data exposure
2	SQL injection: Authentication bypass	Unauthorized access
3	Reflected Cross-Site Scripting (XSS)	Client-side script execution
4	Stored Cross-Site Scripting (XSS)	Persistent client-side script execution
5	Username Enumeration	Account discovery and increased risk of credential attacks
6	Two-Factor Authentication (2FA) Bypass	Unauthorized account access
7	Unprotected Admin Functionality	Unauthorized administrative access
8	Unprotected Admin Functionality with Unpredictable URL	Unauthorized administrative access
9	Path Traversal	Unauthorized file disclosure

10	OS Command Injection	Unauthorized operating-system command execution
11	Blind OS Command Injection with Time Delays	Unauthorized operating-system command execution

DETAILED FINDINGS

Each lab is documented with a description, testing process, key findings, evidence, and recommendations.

LAB 1 – SQL INJECTION: PRODUCT RETRIEVAL

Vulnerability Type: SQL Injection

Objective: Retrieve hidden products by manipulating the SQL query.

Testing Process: Modify the product ID parameter in the URL using a SQL injection payload.

Payload Used: 'OR +1=1—

Key Finding: The application returned additional products that were not normally visible, confirming a SQL injection vulnerability.

Impact: An attacker could retrieve sensitive or hidden data from the database.

Recommendation: Use parameterized queries (prepared statements) and input validation to prevent SQL injection.

LAB 2 – SQL INJECTION: AUTHENTICATION BYPASS

Vulnerability Type: SQL Injection

Objective: Bypass the login authentication and access the administrator account.

Testing Process: Enter the SQL injection payload in the username field.

Payload Used: administrator'--

Key Finding: Successfully bypassed the login form and gained access to the administrator account.

Impact: An attacker could gain unauthorized access to privileged accounts and perform administrative actions.

Recommendation: Implement parameterized queries and proper input validation for all user-supplied data.

LAB 3 – REFLECTED CROSS-SITE SCRIPTING (XSS)

Vulnerability Type: Reflected XSS

Objective: Execute a script in the web browser using the search function.

Testing Process: Insert a script payload into the search box.

Payload Used: <script>alert(1)</script>

Key Finding: The application reflected the input and executed the script, displaying an alert message.

Impact: An attacker could execute malicious scripts in a user's browser, potentially stealing cookies or session tokens.

Recommendation: implement input validation and output encoding to prevent script execution.

LAB 4 – STORED CROSS-SITE SCRIPTING (XSS)

Vulnerability Type: Stored XSS

Objective: Inject a persistent script through the comment functionality.

Testing Process: Submitted a comment with a script payload.

Payload Used: `<script>alert(1)</script>`

Key Finding: The script was stored and executed when viewing the blog post, affecting all users who access the page.

Impact: Persistent XSS can be used to steal session cookies, deface content, or deliver malicious payloads to multiple users.

Recommendation: validate and sanitize user input and encode output before rendering. Additional measures to help prevent XSS include:

- Filter input on arrival.
- Encode data on output.
- Use appropriate response headers.
- Implement a Content Security Policy (CSP).

LAB 5 – AUTHENTICATION FLAWS: USERNAME ENUMERATION VIA DIFFERENT RESPONSES

Vulnerability Type: Authentication weakness/username enumeration

Objective: Identify a valid username based on differences in the application's login responses, then test the account's resistance to password guessing.

Testing Process: Burp Suite was used to send multiple login attempts and compare the application's responses. The differences in the responses made it possible to determine which username was valid. Passwords were then tested against the valid username until the correct credentials were identified.

Payload Used: Provided usernames and passwords list

Key Finding: The application provided distinguishable responses for valid and invalid usernames. This disclosed whether a particular account existed and made subsequent password attacks easier.

Impact: An attacker could enumerate legitimate accounts and focus password guessing or credential attacks specifically on known users, increasing the likelihood of unauthorized account access.

Recommendation: Use consistent authentication error messages for valid and invalid usernames. Implement rate limiting and account lockout controls to reduce the effectiveness of automated username enumeration and password-guessing attempts.

LAB 6 – AUTHENTICATION FLAWS: TWO-FACTOR AUTHENTICATION (2FA) SIMPLE BYPASS

Vulnerability Type: Authentication flaw/ Two-Factor Authentication (2FA) bypass

Objective: Access the victim's account page despite not having access to the victim's 2FA verification code.

Testing Process: Authenticated using the credentials supplied by the lab. After reaching the 2FA verification step, manually navigated to the authenticated account page rather than submitting a valid verification code. The application allowed access without confirming the 2FA had been successfully completed.

Payload/Technique Used: Direct navigation to /my-account

Key Finding: The application allowed direct access to an authenticated account page after the first authentication step without verifying that the second authentication factor had been successfully completed.

Impact: An attacker who obtains a user's name and password could bypass the intended second authentication factor and gain unauthorized access to the victim's account

Recommendation: Enforce 2FA validation server-side before establishing a fully authenticated session or granting access to protected resources. Every protected endpoint should verify that the complete authentication process has been successfully completed rather than relying on the user following the intended page sequence.

LAB 7 – ACCESS CONTROL VULNERABILITIES LAB: UNPROTECTED ADMIN FUNCTIONALITY

Vulnerability Type: Broken access control - Unprotected Administrative Functionality

Objective: Identify and access an administrative panel and delete the designated test user (carlos).

Testing Process: The application's robots.txt file was accessed by appending /robots.txt to the lab URL. The Disallow directive revealed the location of the administrative panel. The disclosed path was then entered into the URL, providing access to the admin panel without administrator authentication. The designated test user was deleted to demonstrate the impact of vulnerability.

Payload/Resource Used: /robots.txt **revealed** /administrator-panel

Key Finding: The application failed to enforce authorization on its administrative functionality. An unauthenticated user who discovered the admin endpoint could access privileged administrative features.

Impact: An unauthorized user could potentially perform privileged actions, including modifying or deleting user accounts and application data.

Recommendation: Access control vulnerabilities can be prevented by taking a defense-in-depth approach and applying the following principles:

- Never rely on obfuscation alone for access control.
- Unless a resource is intended to be publicly accessible, deny access by default.
- Wherever possible, use a single application-wide mechanism for enforcing access controls.
- At the code level, make it mandatory for developers to declare the access that is allowed for each resource, and deny access by default.
- Thoroughly audit and test access controls to ensure they work as designed.

LAB 8 – ACCESS CONTROL VULNERABILITIES LAB: UNPROTECTED ADMIN FUNCTIONALITY WITH UNPREDICTABLE URL

Vulnerability Type: Broken access control - Unprotected Administrative Functionality

Objective: Locate an administrative panel at an unpredictable URL, access the panel without administrator authorization, and delete the designated test user (carlos).

Testing Process: The application's home-page source was reviewed to identify information that could disclose the administrative interface. JavaScript within the page source revealed the otherwise unpredictable URL of the admin panel. Navigating directly to the disclosed URL provided access to administrative functionality without requiring administrator privileges. The designated test user was then deleted to demonstrate the vulnerability.

Payload/Resource Used: JavaScript/page source containing the administrative panel URL

Key Finding: The application attempted to protect administrative functionality by placing it at an unpredictable URL. However, the URL was disclosed within client-side JavaScript, and the server did not properly restrict unauthorized access to the admin panel.

Impact: An unauthorized user who discovers the administrative URL could access privileged functionality and perform administrative actions, such as deleting user accounts.

Recommendation: Access control vulnerabilities can be prevented by taking a defense-in-depth approach and applying the following principles:

- Never rely on obfuscation alone for access control.
- Unless a resource is intended to be publicly accessible, deny access by default.
- Wherever possible, use a single application-wide mechanism for enforcing access controls.
- At the code level, make it mandatory for developers to declare the access that is allowed for each resource, and deny access by default.
- Thoroughly audit and test access controls to ensure they work as designed.

LAB 9 – PATH TRAVERSAL: FILE PATH TRAVERSAL, SIMPLE CASE

Vulnerability Type: Path Traversal/ Directory Traversal

Objective: Exploit a path traversal vulnerability in the application's product image functionality to retrieve the contents of the `/etc/passwd` file.

Testing Process: A request used by the application to retrieve a product image intercepted using Burp Suite. The filename parameter was modified to traverse outside the intended image directory and request `/etc/passwd`. The application's response returned the contents of the requested system file, confirming the vulnerability.

Payload Used: `../../../../etc/passwd`

Key Finding: The application accepted directory traversal sequences within the file name parameter without adequately restricting the requested file location. This allowed access to a file outside the application's intended directory.

Impact: An attacker could possibly retrieve sensitive files accessible to the application process, resulting in unauthorized disclosure of system or application information.

Recommendation: Avoid passing user-supplied input directly to file system APIs. Where user-controlled filenames are necessary, validate them against an allowlist of permitted values and ensure that the resolved file path remains within the application's intended directory.

LAB 10 – OS COMMAND INJECTION, SIMPLE CASE

Vulnerability Type: OS Command injection

Objective: Determine whether the application's product stock checker improperly passes user-controlled input to an operating-system command and demonstrate the vulnerability by identifying the application's current OS user.

Testing Process: Burp Suite was used to intercept a request generated by the product stock-checking functionality. The **storeId** parameter was modified to include an additional operating-system command. After forwarding the modified request, the application's response contained the output of the injected command, confirming that user-controlled input could be executed by the underlying operating system.

Payload Used: 1|whoami

Key Finding: The application incorporated the user-controlled storeId value into a shell command without adequate protection. This allowed an additional OS command to execute on the application server.

Impact: Successful OS command injection could allow an attacker to execute unauthorized commands with the privileges of the application's operating-system user. Depending on those privileges, this could potentially expose sensitive information, modify application data, or compromise the underlying server.

Recommendation: Avoid invoking operating-system commands with user-controlled input whenever possible. Where OS commands are unavoidable, use safe APIs, strictly validate input against an allowlist of expected values, and ensure the application runs with the minimum privileges necessary.

LAB 11 – OS COMMAND INJECTION WITH TIME DELAYS

Vulnerability Type: Blind OS Command injection

Objective: Determine whether user-supplied data within the application's feedback functionality could cause execution of operating-system commands, even though command output was not returned in application's response.

Testing Process: Burp Suite was used to intercept the request generated when submitting feedback. The email parameter was modified to include a command designed to introduce a 10-second delay. After submitting the modified request, the application's response was delayed by approximately 10 seconds, demonstrating that the injected operating-system command had executed.

Payload Used: email=x||ping+-c+10+127.0.0.1||

Key Finding: The application passed user-controlled input from the feedback form into a shell command without adequate validation or isolation. Although the command output was not displayed, the controlled response delay confirmed successful command execution.

Impact: A blind command injection flaw can still allow unauthorized operating-system commands to execute on the server. Even when output is hidden, an attacker may be able to infer information through timing or other side effects and potentially perform more serious actions with the application's privileges.

Recommendation: Avoid passing user-controlled input to operating-system commands. Use safer application-level APIs where possible. If shell execution is unavoidable, strictly allowlist expected

input values, avoid shell interpretation, and run the application with the minimum privileges required.

RECOMMENDATIONS

Based on the findings from the 11 labs, the following general recommendations should be implemented to improve the security of web applications:

1. Use parameterized queries to prevent SQL injection.
2. Validate and sanitize user input on both the client and server sides.
3. Implement secure authentication controls, consistent login responses, and protection against automated password guessing.
4. Implement strict access controls and follow the principle of least privilege.
5. Use output encoding to prevent cross-site scripting (XSS).
6. Enforce multi-factor authentication (MFA) server-side before granting access to protected resources.
7. Restrict and validate file paths to prevent path traversal.
8. Avoid executing operating-system commands with user-controlled input.
9. Apply deny-by-default authorization to sensitive and administrative functionality.
10. Run applications and system processes with the minimum privileges necessary.
11. Conduct regular security testing, code reviews, and vulnerability scanning.

CONCLUSION

The PortSwigger Web Security Academy labs provided valuable hands-on experience with common web application vulnerabilities. Successfully completing these labs demonstrated the importance of secure coding practices and the real-world impact that seemingly simple vulnerabilities can have. By applying the recommended security measures, organizations can significantly reduce the risk of exploitation and better protect their web applications and users.